FACE DETECTION AND

RECOGNITION

Develop an AI application that can detect and recognize faces in images or videos. Use pre-trained face detection models like Haar cascades or deep learning-based face detectors, and optionally add face recognition capabilities using techniques like Siamese networks or ArcFace.

Step 1: Face Detection using Haar Cascades (OpenCV)

We will begin by implementing face detection using OpenCV's Haar Cascade classifiers. This method is efficient and widely used for real-time face detection.

```
!pip install face_recognition opencv-python
```

```
Requirement already satisfied: face_recognition in /usr/local/lib/python3.12/dist-packages (1.3.0)
Requirement already satisfied: opencv-python in /usr/local/lib/python3.12/dist-packages (4.13.0.92)
Requirement already satisfied: face-recognition-models>=0.3.0 in /usr/local/lib/python3.12/dist-packages (from face_recognit
Requirement already satisfied: Click>=6.0 in /usr/local/lib/python3.12/dist-packages (from face_recognition) (8.3.1)
Requirement already satisfied: dlib>=19.7 in /usr/local/lib/python3.12/dist-packages (from face_recognition) (19.24.6)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from face_recognition) (2.0.2)
Requirement already satisfied: Pillow in /usr/local/lib/python3.12/dist-packages (from face_recognition) (11.3.0)
```

```
import cv2
import face_recognition
import numpy as np
from google.colab.patches import cv2_imshow
from google.colab import files
```

```
uploaded = files.upload()
```

Choose Files pic.avif

pic.avif(image/avif) - 13930 bytes, last modified: 4/14/2026 - 100% done

```
Saving pic.avif to pic (1).avif
```

```
known_encodings = []
known_names = []

img1 = face_recognition.load_image_file("pic.avif")
enc1 = face_recognition.face_encodings(img1)[0]
known_encodings.append(enc1)
known_names.append("Person 1")

print("✅ Known faces encoded!")
```

```
✅ Known faces encoded!
```

```
uploaded = files.upload()
```

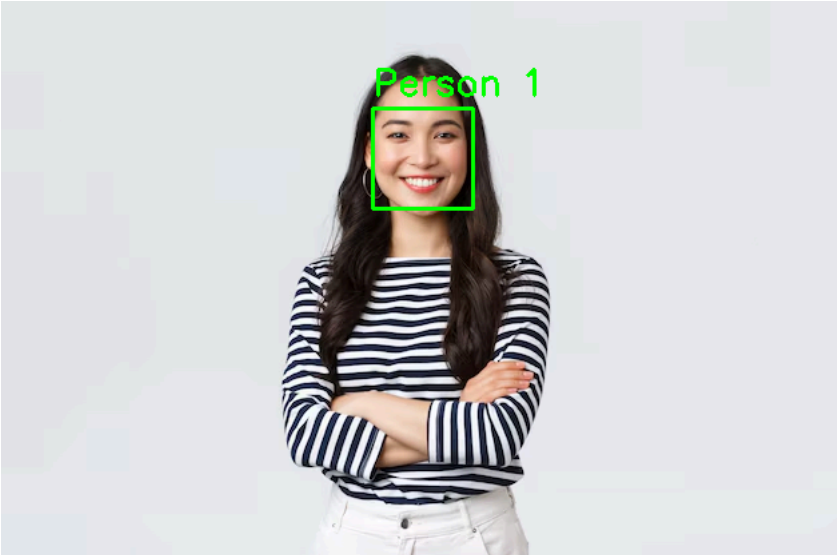
Choose Files pic.avif

pic.avif(image/avif) - 13930 bytes, last modified: 4/14/2026 - 100% done

```
Saving pic.avif to pic (3).avif
```

```
image = face_recognition.load_image_file("pic.avif")

rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

face_locations = face_recognition.face_locations(rgb)

face_encodings = face_recognition.face_encodings(rgb, face_locations)

for (top, right, bottom, left), face_encoding in zip(face_locations, face_encodings):

    matches = face_recognition.compare_faces(known_encodings, face_encoding)
    name = "Unknown"

    face_distances = face_recognition.face_distance(known_encodings, face_encoding)
    best_match_index = np.argmin(face_distances)

    if matches[best_match_index]:
        name = known_names[best_match_index]

    cv2.rectangle(rgb, (left, top), (right, bottom), (0, 255, 0), 2)
```

```
    cv2.putText(rgb, name, (left, top - 10),
                cv2.FONT_HERSHEY_SIMPLEX, 0.9, (0, 255, 0), 2)

cv2_imshow(rgb)
```




```
from IPython.display import display, Javascript
from google.colab.output import eval_js
import base64

def take_photo(filename='pic.avif', quality=0.8):
    js = Javascript('''
    async function takePhoto(quality) {
      const div = document.createElement('div');
      const video = document.createElement('video');
      const stream = await navigator.mediaDevices.getUserMedia({video: true});
      document.body.appendChild(div);
      div.appendChild(video);
      video.srcObject = stream;
      await video.play();

      await new Promise(resolve => setTimeout(resolve, 2000));

      const canvas = document.createElement('canvas');
      canvas.width = video.videoWidth;
      canvas.height = video.videoHeight;
      canvas.getContext('2d').drawImage(video, 0, 0);

      stream.getTracks().forEach(track => track.stop());
      div.remove();

      return canvas.toDataURL('image/jpeg', quality);
    }
    ''')
    display(js)
    data = eval_js('takePhoto({})'.format(quality))
    binary = base64.b64decode(data.split(',')[1])
    with open(filename, 'wb') as f:
        f.write(binary)
    return filename
```

```
filename = take_photo()
print("Saved:", filename)
```

```
Saved: pic.avif
```